

EcoDB — Retrieval Benchmark Progression

Internal engineering log · how the retrieval engine was measured, found wanting, and fixed. Author / architect: José Antonio Ortiz Melo (Eco Consulting). Real numbers, no framing.

The setup

EcoDB is a multi-agent long-term memory system (PostgreSQL + pgvector + Apache AGE + Jina v4 embeddings + GLiNER + MCP). Retrieval runs through GAMR, a multi-stage engine that composes vector similarity, BM25 lexical search, knowledge-graph expansion, temporal freshness, contradiction detection and a cross-encoder reranker. I evaluate it two ways: against a **locked internal golden set of 243 queries** (5 categories: factual, analytical, historical, contextual, cross_modal — ground truth frozen across every experiment), and against **LoCoMo** (ACL 2024; 1,982 queries) for comparison with published systems. This log is the honest version of what those numbers taught me.

1. The uncomfortable finding: my engine was making things worse

For the first time I ran pure cosine retrieval (ANN, no GAMR) as a control — something I should have done on day one. On the first 63-query slice, GAMR looked like it helped (analytical R@5 +0.141). Then I expanded to the full 243-query set, and the win evaporated: **GAMR was net-negative in every single category**. The 63-query result had been overfitting.

Category	n	ANN R@5	GAMR R@5	Δ R@5	ANN MRR	GAMR MRR	Δ MRR
factual	65	0.528	0.449	-0.079	0.441	0.297	-0.144
analytical	52	0.603	0.542	-0.061	0.462	0.427	-0.035
historical	46	0.399	0.388	-0.011	0.326	0.269	-0.057
contextual	51	0.477	0.379	-0.098	0.394	0.254	-0.140
cross_modal	29	0.672	0.603	-0.069	0.534	0.402	-0.132
OVERALL	243	0.526	0.461	-0.065	0.425	0.323	-0.102

R@10 was identical for ANN and GAMR — GAMR found no memory cosine didn't already find. It only reordered within the same ceiling, and reordered *badly*. The sophisticated part of my system was pure cost.

2. Root cause: structure, not weights

The composite score was **additive**, and semantic similarity was only 25–35% of it. The other 65–75% were signals that, at that moment, didn't work:

Signal	Weight	State	Verdict
semantic (cosine)	25-35%	Jina v4 works well	REAL
bm25	15-20%	Lifts wrong keyword matches in factual / historical	DAMAGE
freshness	10-25%	Corpus temporally concentrated, all recent	NOISE
graph	10-25%	633 nodes, ~0 useful edges (GLiNER extracts entities, not relations)	~ZERO
weight	15-25%	decisions=0.9 > observations=0.5	SOME

Concretely, for a factual query the correct memory (cosine 0.72) lost the top slot to a more recent memory with matching keywords (BM25 0.80, freshness 0.90) because the additive composite rewarded the wrong thing. That is the -0.144 factual MRR collapse, explained.

3. The fix: embedding is truth, everything else is opinion

The insight that turned it around was structural, not numeric: GAMR should not *replace* cosine, it should *modulate* it. I switched the composite from additive to multiplicative:

`composite = semantic × (1.0 + bm25×0.10 + graph×0.05 + weight×0.05 + freshness×0.02)`

Now opinion can lift truth but never overwrite it. On the same 243-query set, GAMR finally beat pure cosine: overall MRR **0.431 vs 0.425**, analytical MRR **+0.067**, historical +0.032 R@5 / +0.033 MRR, contextual back to parity (from -0.098). A one-line change in score structure did what no amount of weight tuning could.

4. Then I tuned it to death — and stopped

Ten tuning experiments on the locked 243-query golden set, sweeping coefficients and re-running after a graph densification (589 active nodes, dictionary 17→585 entries, +146% entity links):

Experiment	R@5	MRR
EXP-7 production baseline (AGE-sync + BM25-gating)	0.524	0.411
EXP-8a BM25 coeff 0.30	0.524	0.394
EXP-8b weight coeff 0.30	0.524	0.392
EXP-9a post-densification graph=0.05	0.520	0.391
EXP-9b post-densification graph=0.15	0.520	0.372

Conclusion I wrote down and acted on: **current coefficients are optimal; amplifying any signal hurts MRR; graph_score contributes to no coefficient**. I capped graph_bonus at 0.05 and stopped adding knobs. Knowing when to stop tuning is part of the job.

5. The real lever was the corpus, not the engine

While diagnosing the engine, one cheap intervention beat all of it: indexing 25 documents and 13 memories that existed in the workspace but not in the index.

Intervention	Δ R@5	Effort
Adversarial bug fixes (19 bugs)	+0.043	1 session
Full GAMR (BM25 + weights + graph)	+0.034	3 phases
Corpus (index existing docs)	+0.075	~2 hours

The biggest retrieval gain came from the least sophisticated work. I keep this table because it's the honest order of priorities for a memory system: corpus completeness first, engine second.

6. The breakthrough: ingestion granularity, not ranking

On LoCoMo (1,982 queries) I tested the cross-encoder reranker (ms-marco-MiniLM-L-6-v2, ~22M params) against chunking (Docling, 5-turn sliding windows, 1-turn overlap). The reranker lifted R@1 by +0.164 but was not the lever — the bottleneck was retrieval, and chunking was the answer:

Config	R@1	R@5	R@10
Baseline (day 78, no reranker)	0.4141	0.7689	0.8940
+ P1 cross-encoder reranker	0.5782	0.7926	0.8693
Chunked, K=20 (reranker OFF)	0.7759	0.9273	0.9618
Chunked, K=5 (reranker OFF)	0.7742	0.9141	0.9141

Chunking alone took R@5 from 0.769 to **0.927** (+19.9%), beating the 0.90 target with the reranker off. Lesson carried forward: **for conversational data, ingestion granularity drives Recall@5; ranking sophistication drives Recall@1**. They optimize different things, and the reranker costs latency — so you spend it only when R@1 is what the use case needs.

7. External context

Where EcoDB's retrieval lands against published memory systems on LoCoMo:

System	Metric	Score	Notes
ByteRover 2.0	accuracy (LLM-judge)	92.2%	end-to-end (retrieval + generation)
MemMachine v0.2	LLM-score	91.7%	gpt-4-mini
Hindsight (Gemini)	accuracy	89.6%	entity-temporal memory
EcoDB (chunked, K=20)	Recall@5	92.7%	retrieval only, no LLM generation
GPT-4 (baseline paper)	F1	32.1	original baseline

Note the asymmetry: the systems above EcoDB measure end-to-end accuracy with an LLM in the loop; EcoDB's 92.7% is retrieval alone. Different metrics, shown honestly — I'm not claiming a ranking, I'm placing the number in context.

What this cost, and what I take from it

- Run the dumb control early. The cosine-only test should have been day one; skipping it let me build sophistication on an unverified assumption.
- Be willing to find your best work is net-negative, on a locked golden set you can't game.
- Score *structure* (additive vs multiplicative) mattered more than any weight.
- Corpus completeness beat engine sophistication; ingestion granularity beat ranking.
- Know when to stop tuning — ten experiments said the knobs were done.

Shared as a work artifact: internal engineering documentation, not a presentation. All numbers are from EcoDB's benchmark logs against a locked golden set and LoCoMo; evaluation scripts live in `eval/`, runs are reproducible and diffable, no queries excluded.